PROJECT DOCUMENTATION

Document Q&A Chatbot Using RAG

With Evaluation Harness for Retrieval Hit Rate and Answer Quality

Document type	Technical project documentation
Design	High-Level Design (HLD) and Low-Level Design (LLD)
Core approach	Retrieval-Augmented Generation (RAG)
Evaluation	Hit Rate@K, Recall@K, MRR, Precision@K, faithfulness, answer relevance
Suggested stack	Python, Streamlit, Sentence Transformers, FAISS/ChromaDB, LLM, RAGAS

Includes dedicated, full-width architecture diagrams embedded as images. Example scores are illustrative; actual performance must be measured using the implemented benchmark.

Contents

1. Project Overview
2. High-Level Architecture (HLD)
3. Low-Level Architecture (LLD)
4. RAG Pipeline and Working
5. Evaluation Harness and Metrics
6. Technology Stack
7. Project Folder Structure
8. Testing and Validation
9. Security and Reliability
10. Deliverables and Future Enhancements

1. Project Overview

Abstract. The Document Q&A Chatbot enables users to upload PDF, DOCX, and TXT files and ask natural-language questions about their contents. It uses Retrieval-Augmented Generation (RAG): relevant passages are retrieved from uploaded documents and supplied to a Large Language Model (LLM) to generate evidence-grounded answers with source references.

Its distinguishing feature is an evaluation harness that measures retrieval and answer quality against a fixed, labeled benchmark. It supports repeatable experiments, failure analysis, configuration comparison, and regression testing.

Problem statement

Manual document review is time-consuming, and general-purpose chatbots may produce unsupported answers or miss relevant passages. The proposed system combines semantic retrieval, context-grounded generation, citations, and automated evaluation to make document question answering more verifiable and measurable.

Objectives

• Support document uploads and natural-language questions.
• Implement parsing, chunking, embeddings, vector search, and LLM generation.
• Display source document and page/passage references.
• Calculate Hit Rate@K, Recall@K, MRR, and Precision@K.
• Evaluate faithfulness, relevance, and correctness.
• Export evaluation reports and compare configurations.

Scope

The initial release supports PDF, DOCX, and TXT, semantic retrieval, source citations, and offline benchmark evaluation. Hybrid retrieval, reranking, multi-user authentication, OCR, and deployment are extensions.

2. High-Level Architecture (HLD)

The HLD shows the major system components and their interactions. The system has two workflows: interactive question answering and offline benchmark evaluation.

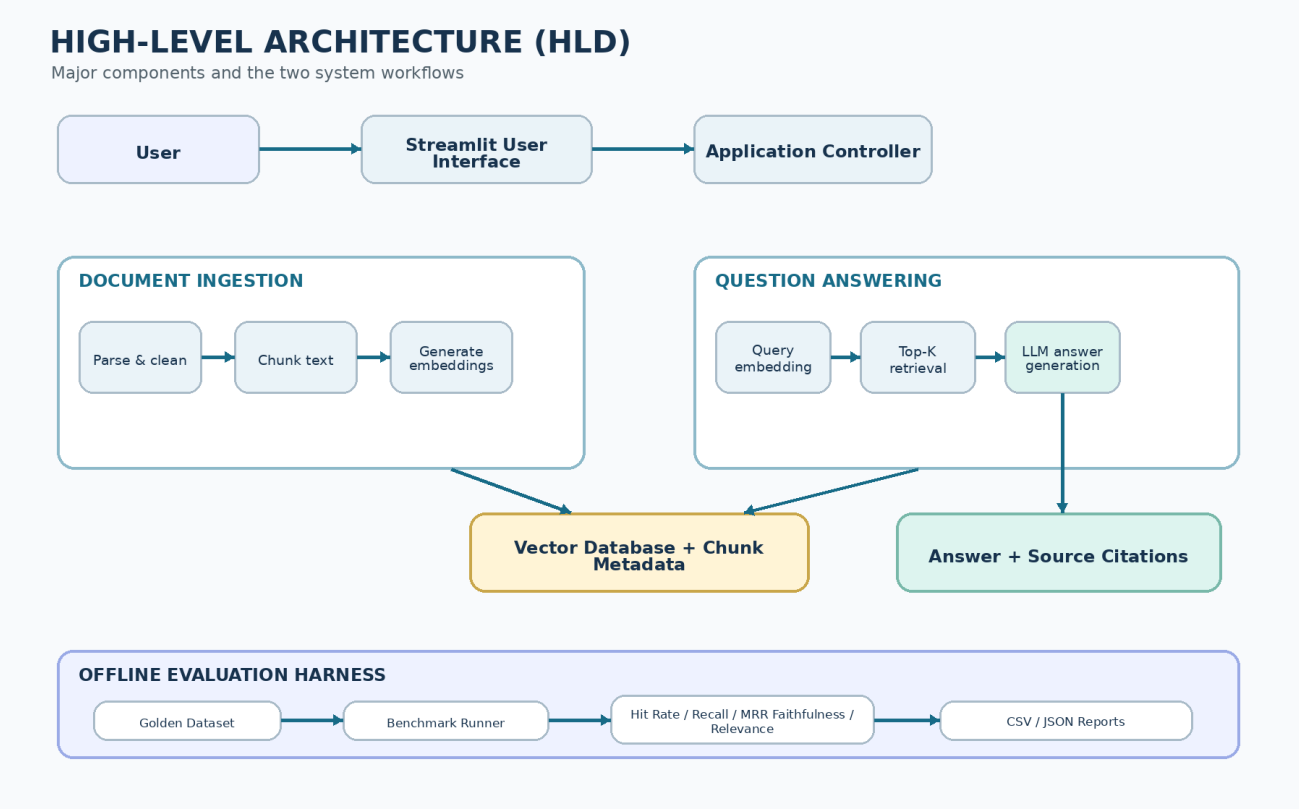


Figure 1. High-level architecture diagram (embedded image).

Component	Responsibility
User interface	Accept uploads/questions; show answers, sources, and evaluation scores.
Application controller	Coordinate ingestion, chat requests, sessions, and errors.
Ingestion pipeline	Extract text, clean it, chunk it, and create embeddings.
Vector database	Store embeddings and metadata; return similar passages.
RAG retrieval pipeline	Embed the query and retrieve top-K passages.
LLM answer generator	Generate answers grounded in retrieved context.
Evaluation harness	Run benchmark questions through a fixed pipeline.
Golden dataset	Store reference answers and relevant chunk IDs.
Metrics and reports	Calculate metrics and export results.

HLD workflow A — Question answering

1. Upload documents.
2. Extract text and create chunks.
3. Store chunk embeddings and metadata.
4. Embed the question and retrieve relevant passages.
5. Generate an answer from retrieved context.
6. Display the answer and supporting sources.

HLD workflow B — Evaluation

1. Load a fixed benchmark dataset.
2. Run each question through retrieval.
3. Compare retrieved IDs with relevant IDs.
4. Calculate retrieval metrics.
5. Generate and evaluate answers.
6. Export scores and failures.

3. Low-Level Architecture (LLD)

The LLD shows the internal modules, storage interfaces, and evaluation flow. Ingestion, retrieval, generation, and evaluation should remain separate so each can be tested independently.

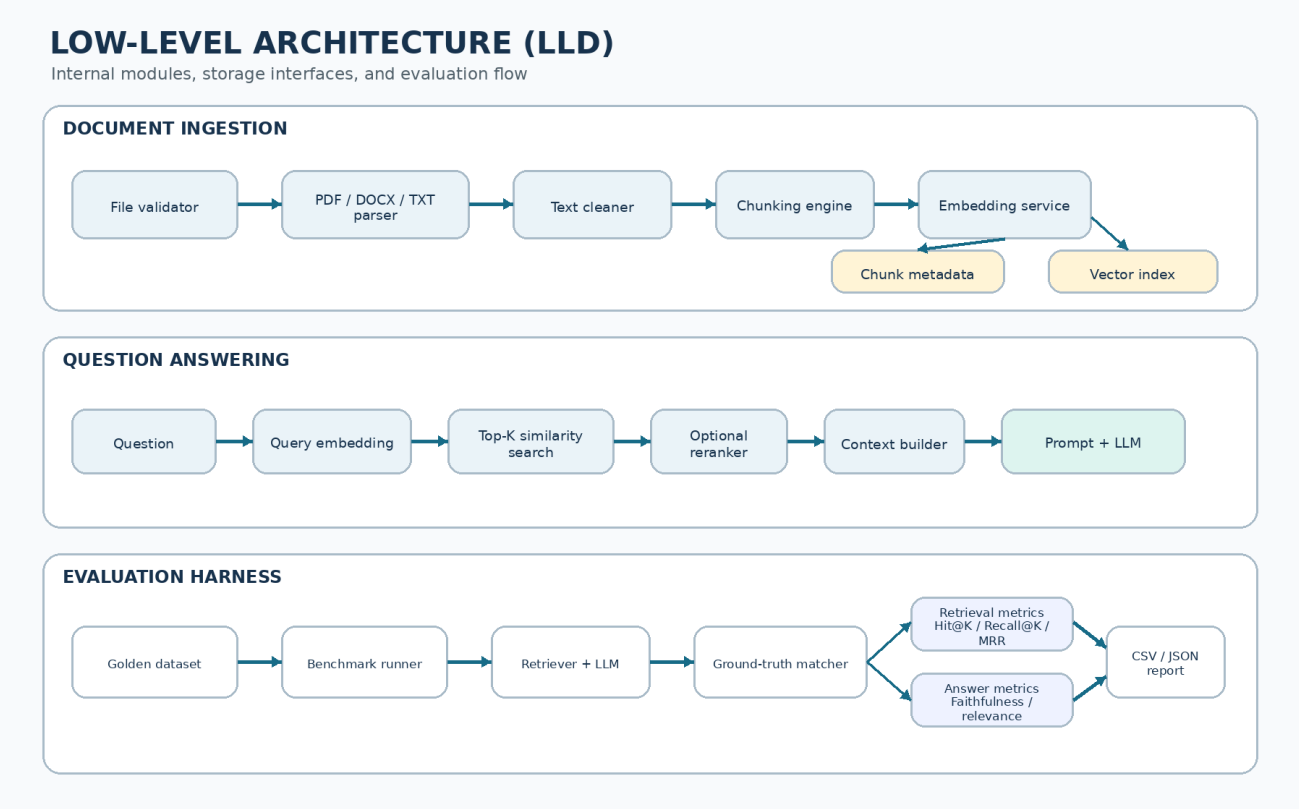


Figure 2. Low-level architecture diagram (embedded image).

Module 1 — File validator

Validate extensions and file size, reject corrupt or empty files, assign a document ID, and track ingestion status.

Module 2 — Document parser

Use PyMuPDF for PDF text extraction, python-docx for DOCX, and standard file reading for TXT. Preserve page numbers where available and handle parsing errors.

Module 3 — Chunking engine

Split text into passages with configurable chunk size and overlap. Store chunk ID, document ID, source name, page number, chunk index, and text.

Example chunk record

```
{
  "chunk_id": "doc01_chunk_004",
  "document_id": "doc01",
  "document_name": "guide.pdf",
  "page_number": 5,
  "text": "The system retrieves relevant passages...",
  "chunk_index": 4
}
```

Module 4 — Embedding and indexing

Convert each chunk into a dense vector and store it with metadata. Embed queries in the same compatible embedding space and search using the index's supported similarity or distance measure.

Module 5 — Retrieval engine

Retrieve top-K candidates and return IDs, text, scores, and source metadata. An optional reranker can reorder candidates before context construction.

Module 6 — Answer generation

Construct a prompt from the question and retrieved passages. Require evidence-grounded answers and source references; instruct the model to abstain when evidence is insufficient. Prompting does not guarantee hallucination-free output.

Module 7 — Evaluation engine

Load benchmark cases, run retrieval with fixed settings, calculate deterministic retrieval metrics from annotated chunk IDs, generate answers, evaluate answer quality, and save per-question results.

4. RAG Pipeline and Working

4.1 Document ingestion

Workflow: upload → validate → parse → clean → chunk → embed → store vectors and metadata. Chunk size and overlap affect context completeness, retrieval precision, and processing cost.

4.2 Question answering

Embed the question, search for similar chunks, optionally rerank candidates, assemble context, invoke the LLM, and display an answer with document/page citations.

4.3 Example answer prompt

```
You are a document question-answering assistant.
Answer using only the supplied context.
Do not invent facts absent from the context.
If the context is insufficient, say so.

Question:
{question}

Retrieved context:
{context}

Return a concise answer and source references.
```

Treat this prompt as a baseline. Evaluate answers instead of assuming instructions alone prevent unsupported claims.

5. Evaluation Harness and Metrics

The evaluation harness is the project's main differentiator. It separates retrieval performance from answer-generation quality so failures can be diagnosed at the correct stage.

5.1 Golden evaluation dataset

Create a fixed dataset containing questions, reference answers, and ground-truth relevant chunk IDs. The IDs must match chunks produced by ingestion. If chunking changes, update or remap the annotations.

```
[
  {
    "question_id": "Q001",
    "question": "What is the main objective of the system?",
    "reference_answer": "Retrieve relevant information and answer questions.",
    "relevant_chunk_ids": ["doc01_chunk_002", "doc01_chunk_003"]
  }
]
```

This is an illustrative record. Build and manually verify the real dataset against source documents.

5.2 Retrieval metrics

Let N be the number of benchmark questions, R■■ the top-K retrieved chunk IDs, G■ the ground-truth relevant chunk IDs, and r■ the rank of the first relevant result.

Hit Rate@K

HitRate@K = (1/N) × Σ■ ■[R■■ ∩ G■ ≠ ∅]

Fraction of questions for which at least one relevant chunk appears in the top K. For example, 85 hits out of 100 gives Hit Rate@5 = 85%.

Recall@K

Recall@K = (1/N) × Σ■ |R■■ ∩ G■| / |G■|

Average fraction of annotated relevant chunks retrieved. Hit rate checks whether any relevant chunk was found; recall checks how many relevant chunks were found.

Mean Reciprocal Rank (MRR)

MRR = (1/N) × Σ■ 1/r■

Measures how early the first relevant result appears. If no relevant result is retrieved, that question contributes zero.

Precision@K

Precision@K = (1/N) × Σ■ |R■■ ∩ G■| / K

Fraction of the top-K results that are relevant, assuming K results are returned for every question.

Implementation note: Calculate retrieval metrics from annotated relevance labels, not LLM-generated relevance judgments. Define handling of duplicates, missing labels, and fewer-than-K results.

5.3 Answer-quality metrics

Metric	What it evaluates
Faithfulness	Whether answer claims are supported by retrieved context.
Answer relevance	Whether the answer addresses the question.
Answer correctness	Agreement with reference answers and verified facts.
Context precision	Whether retrieved passages are predominantly relevant.
Context recall	Whether context contains the evidence needed to answer.

RAGAS can support semantic answer/context evaluation. LLM-based judgments are estimates, not ground truth; manually review a sample and record evaluator configuration.

5.4 Evaluation execution

1. Load the golden dataset.
2. Load a fixed pipeline configuration.
3. Retrieve top-K chunks.
4. Compare retrieved IDs with ground truth and calculate retrieval metrics.
5. Generate answers and evaluate answer quality.
6. Save per-question and aggregate results.
7. Export CSV/JSON and compare experiments.

5.5 Example result format

Question	Hit@5	Recall@5	MRR	Faithfulness
Q001	1	1.00	1.00	0.95
Q002	1	0.50	0.50	0.90
Q003	0	0.00	0.00	N/A

Scores are illustrative, not measured performance. Use N/A when answer-quality evaluation is not applicable or was not run; do not infer faithfulness solely from retrieval failure.

6. Technology Stack

Layer	Recommended technology	Purpose
Programming language	Python	Core application and evaluation logic.
User interface	Streamlit	Uploads, chat, sources, metric tables, charts.
PDF parsing	PyMuPDF	Text extraction and page tracking.
DOCX parsing	python-docx	Extract document text.
Embeddings	Sentence Transformers	Encode chunks and queries.
Vector store	FAISS or ChromaDB	Index and retrieve embeddings.
Answer generation	Gemini API or local LLM	Generate context-grounded answers.
Evaluation	Custom Python + RAGAS	Retrieval metrics and answer evaluation.
Data handling	NumPy, pandas	Metric calculations and tables.
Testing	pytest	Unit, integration, regression tests.
Visualization	Plotly	Charts and experiment comparisons.
Metadata storage	SQLite (optional)	Document and experiment history.

Recommended starting configuration

Start with Python, Streamlit, Sentence Transformers, FAISS or ChromaDB, an LLM, and custom deterministic retrieval metrics. Add RAGAS once the retrieval benchmark is working.

7. Proposed Project Folder Structure

```
document-qa-rag/
■■■ app.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ src/
■   ■■■ ingestion/
■   ■   ■■■ file_validator.py
■   ■   ■■■ document_parser.py
■   ■   ■■■ chunker.py
■   ■■■ embeddings/embedding_service.py
■   ■■■ retrieval/
■   ■   ■■■ vector_store.py
■   ■   ■■■ retriever.py
■   ■   ■■■ reranker.py
■   ■■■ generation/
■   ■   ■■■ prompt_builder.py
■   ■   ■■■ answer_generator.py
■   ■■■ evaluation/
■   ■   ■■■ dataset_loader.py
■   ■   ■■■ retrieval_metrics.py
■   ■   ■■■ answer_metrics.py
■   ■   ■■■ benchmark_runner.py
■   ■   ■■■ report_generator.py
■   ■■■ storage/metadata_store.py
■■■ evaluation_data/evaluation_dataset.json
■■■ tests/
■   ■■■ test_ingestion.py
■   ■■■ test_retrieval_metrics.py
■   ■■■ test_rag_pipeline.py
■■■ reports/.gitkeep
```

This is a suggested layout, not a claim that the files already exist.

8. Testing and Validation

Unit testing

- Verify parsing for each file type.
- Test chunk IDs, metadata, and page references.
- Check metric formulas against hand-calculated examples.
- Test empty retrieval, missing labels, duplicate results, and malformed inputs.

Integration testing

- Verify the upload-to-answer workflow.
- Confirm retrieved chunk IDs map to correct metadata.
- Check citations and report exports.
- Confirm evaluation runs use the expected model and configuration.

Benchmark testing

- Run a fixed, manually reviewed dataset.
- Compare K values such as 1, 3, and 5.
- Compare chunk sizes, embedding models, and reranking settings.
- Record latency and scores; rerun after changes to detect regressions.

9. Security and Reliability

• Validate file type and size; handle malformed documents safely.
• Store API keys in environment variables and exclude secrets from version control.
• Treat document text as untrusted input, not as instructions to the LLM.
• Isolate documents and vector searches between users if multi-user support is added.
• Delete vectors and metadata when a document is deleted.
• Record model versions and configuration for reproducible evaluation.
• Distinguish evidence-backed answers from uncertain or unsupported responses.

10. Expected Deliverables

1. Working document upload and Q&A application.
2. RAG pipeline with source citations.
3. Vector index and metadata store.
4. Golden benchmark dataset with relevant chunk IDs.
5. Deterministic retrieval metrics implementation.
6. Answer-quality evaluation component.
7. Dashboard with aggregate and per-question results.
8. CSV/JSON exports.
9. Unit and integration tests.
10. Final report with architecture, methodology, measured results, limitations, and future work.

11. Future Enhancements

Potential extensions include hybrid retrieval (BM25 + dense search), cross-encoder reranking, multilingual support, OCR for scanned PDFs, authentication, experiment tracking, and CI-based regression benchmarks.

12. Conclusion

This project combines document question answering with a repeatable evaluation framework. Hit Rate@K, Recall@K, MRR, and Precision@K measure retrieval success, while faithfulness, answer relevance, and correctness assess generated responses. Separating these evaluations helps identify whether failures originate in document processing, retrieval, or generation.

Implementation note: the diagrams and sample scores are illustrative. Report actual performance only after running the benchmark on a documented test dataset.